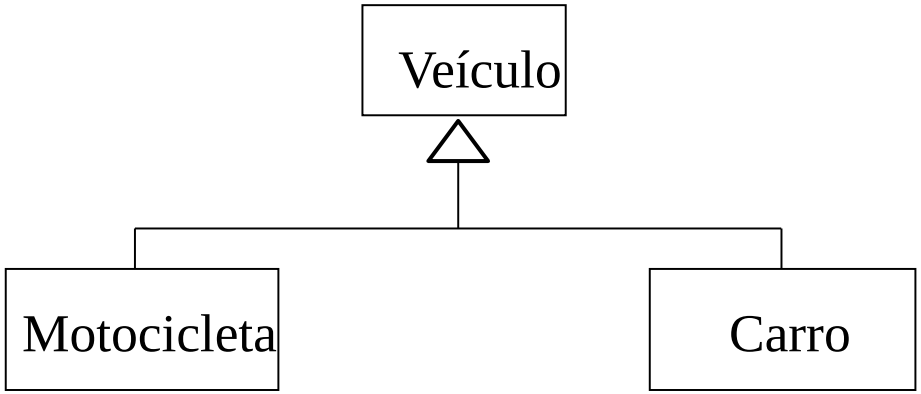
XDES02
Programação Orientada a Objetos

Aula 09: Classes abstratas

Classes abstratas

- Na nossa última aula implementamos o modelo a seguir



- A classe Veículo foi criada para conter as informações comuns de Motocicleta e Carro
- Não faz sentido criar instâncias de Veículo
- Veículo, portanto, deve ser uma classe **Abstrata**
- Classes que não são abstratas, são chamadas **Concretas**

Classes abstratas

- Classe concreta
  - Permite que sejam produzidas instâncias (objetos)
- Classe abstrata
  - Não permite instanciação
  - São implementadas com o objetivo de serem estendidas por subclasses concretas
  - Contém atributos e comportamentos comuns a suas subclasses

Classes abstratas

- Classes abstratas podem conter ou não métodos abstratos, mas se uma classe **possui ao menos um método abstrato**, esta deve ser declarada como abstrata
- Se uma classe é subclasse de uma classe abstrata, então ela é obrigada a implementar todos os seus métodos abstratos
- Método abstrato tem a função de forçar comportamento nas subclasses
  - Obriga a subclasse fornecer uma implementação concreta do método abstrato

Classes abstratas em Python

- Para criar uma classe abstrata em Python, deve-se herdar da meta-classe **ABC** que pertence ao módulo **abc** (Abstract Base Class)

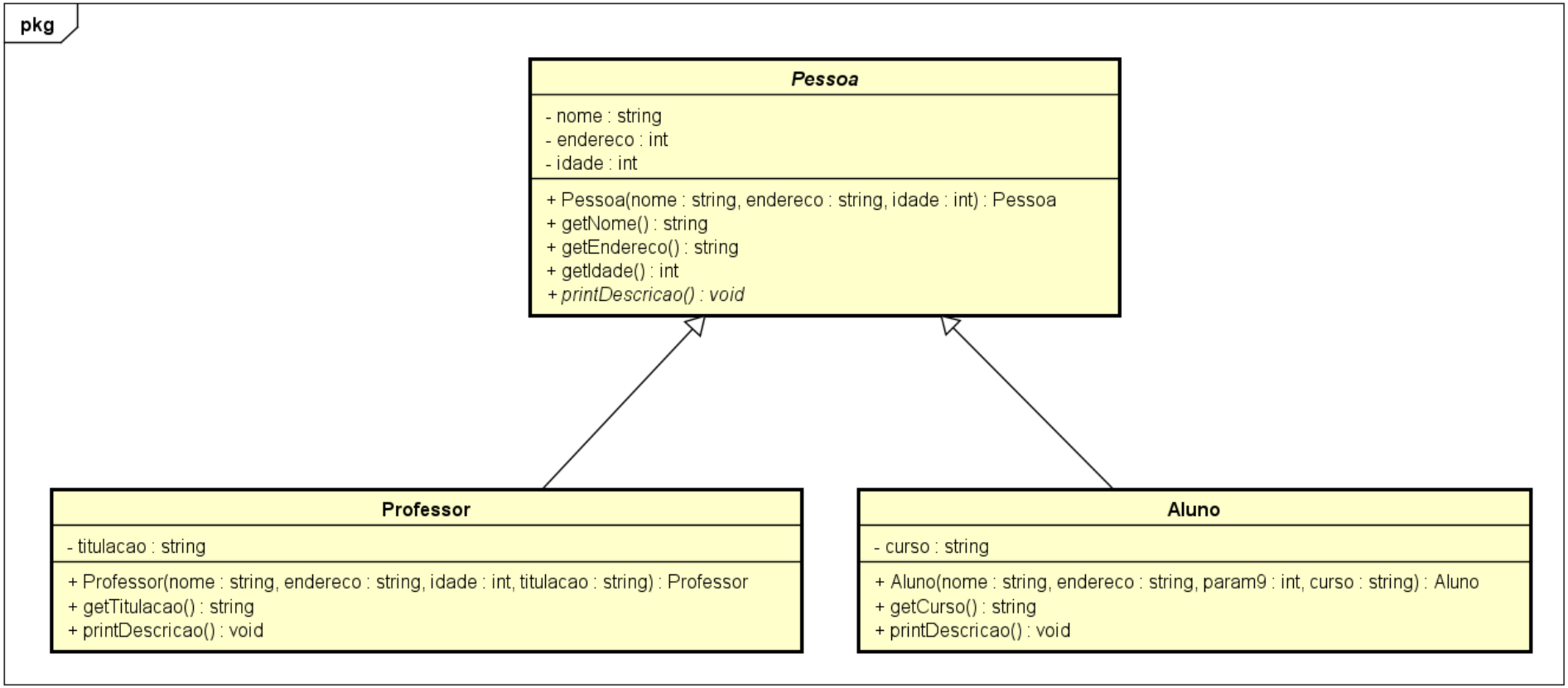
```
from abc import ABC
class MinhaClasse(ABC):
```

- Para definir um método abstrato deve-se importar **abstractmethod** e usar o docorator **@abstractmethod**

```
from abc import ABC, abstractmethod
class MinhaClasse(ABC):
    @abstractmethod
    def meuMetodo(self):
        pass
```

Exercício 1

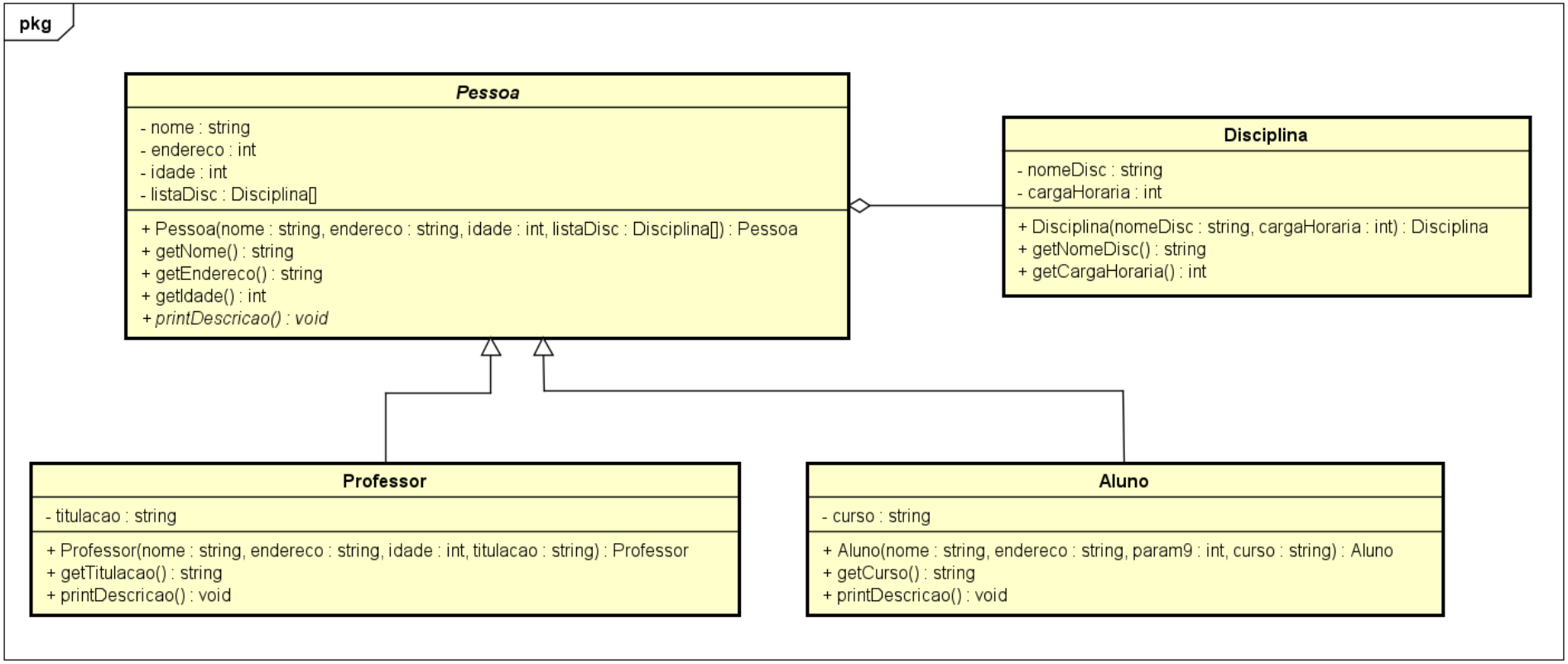
- Vamos criar um sistema acadêmico simplificado



Exercício 2

- Vamos agora complementar nosso modelo incluindo a classe Disciplina
  - Tanto alunos quanto professores possuem relacionamento com disciplina
    - Alunos cursam disciplina
    - Professores ministram disciplina
  - Podemos, então, criar um relacionamento de Disciplina com Pessoa
    - Alunos e Professores vão herdar esse relacionamento

Exercício 2


pkg
Pessoa
- nome : string
- endereco : int
- idade : int
- listaDisc : Disciplina[]
+ Pessoa(nome : string, endereco : string, idade : int, listaDisc : Disciplina[]) : Pessoa
+ getNome() : string
+ getEndereco() : string
+ getIdade() : int
+ printDescricao() : void
Disciplina
- nomeDisc : string
- cargaHoraria : int
+ Disciplina(nomeDisc : string, cargaHoraria : int) : Disciplina
+ getNomeDisc() : string
+ getCargaHoraria() : int
Professor
- titulacao : string
+ Professor(nome : string, endereco : string, idade : int, titulacao : string) : Professor
+ getTitulacao() : string
+ printDescricao() : void
Aluno
- curso : string
+ Aluno(nome : string, endereco : string, param9 : int, curso : string) : Aluno
+ getCurso() : string
+ printDescricao() : void

Exercício 3

- Vamos agora criar o método insereDisc, que permite inserir disciplinas para professores e alunos
- Pergunta:
  - Em qual classe devemos inserir este método?